Oficina IA · Fase 1

El dominio de la oficina, todavía sin IA — enunciado de práctica

Backend · Java 21 · repo `marcosr-se/ai-office` · noviembre 2026

Qué vas a construir

Las clases que representan la oficina (agentes, tareas, mensajes…) y un **programa de consola** que simula un encargo completo de principio a fin:

1. El cliente hace un encargo al CEO.
2. El CEO lo divide en tareas y las reparte entre Developer, Designer y Tester.
3. Cada agente hace su tarea y avisa al CEO con un mensaje.
4. Todos van a la sala de reuniones y se imprime un resumen.

Todavía **no hay IA real**. Las "respuestas de la IA" salen de una clase `MockProvider` que devuelve textos fijos. En la fase 2 cambiarás el Mock por Gemini **cambiando una sola línea**. Por eso en esta fase lo importante es el diseño, no la cantidad de código.

Reglas

- Todo el código va dentro de `backend/src/main/java/com/marcosr_se/backend/` (y los tests en `backend/src/test/java/...`).
- Nombres de clases, métodos y variables **en inglés**.
- Sin Lombok. Los getters los escribes tú o los genera IntelliJ: **Cmd + N → Getter**.
- Las clases de `domain` y `llm` son **Java puro**: nada de `@Component`, `@Service` ni otras anotaciones de Spring.
- Todos los atributos `private`. Pon `final` en todo lo que no cambia después del constructor.
- Nunca devuelvas una lista interna tal cual: devuelve una copia con `List.copyOf(...)`. Así nadie puede modificarla desde fuera.
- Haz **un commit al terminar cada paso**, con mensaje en inglés. Ejemplo: `Add Role enum`.

Estructura de paquetes

Crea estos paquetes dentro de `com.marcosr_se.backend` (al lado del paquete `health` que ya tienes):

```
com.marcosr_se.backend
├── health/         (ya existe)
├── domain/         Role, Location, TaskStatus, Message, Agent, Task, Office
├── llm/            LLMProvider, MockProvider
└── simulation/     Simulation, ConsoleApp

src/test/java/com/marcosr_se/backend
├── domain/         AgentTest, TaskTest, OfficeTest
└── llm/            MockProviderTest
```

PASO 1 Role — los puestos de trabajo (enum)

Crea el enum `Role` en `domain` con estos valores: `CEO`, `DEVELOPER`, `DESIGNER`, `TESTER`.

Cada valor tiene un atributo `displayName` (el nombre bonito para imprimir):

Valor	displayName
`CEO`	"CEO"
`DEVELOPER`	"Developer"
`DESIGNER`	"Designer"
`TESTER`	"Tester"

- Añade el atributo `private final String displayName`, un constructor y el método `getDisplayName()`.

> **Pista**
> Un enum puede tener atributos y constructor. Se escribe así: `DEVELOPER("Developer"),`. El constructor de un enum no lleva `public`.

> **Hecho cuando…**
> `Role.DEVELOPER.getDisplayName()` devuelve `"Developer"`.

PASO 2 Location y TaskStatus — dos enums sencillos

- `Location` en `domain`, con los valores `DESK` (su mesa) y `MEETING_ROOM` (sala de reuniones).
- `TaskStatus` en `domain`, con los valores `PENDING`, `IN_PROGRESS` y `DONE`.
- Estos no llevan atributos ni constructor: solo la lista de valores.

PASO 3 Message — un mensaje entre agentes (record)

Crea el record `Message` en `domain` con tres componentes de tipo `String`:

Componente	Qué es
`from`	id del agente que lo envía (ej. `"ceo"`)
`to`	id del agente que lo recibe (ej. `"dev"`)
`content`	el texto del mensaje

- **Validación:** si alguno es `null` o está vacío, lanza `IllegalArgumentException`.

> **Pista**
> • Un record puede tener un **constructor compacto**: `public Message { ... }` (sin paréntesis). Ahí dentro compruebas los valores y lanzas la excepción.
> • Para ver si un String está vacío: `texto == null || texto.isBlank()`.

Hecho cuando…
`new Message("ceo", "dev", "")` lanza `IllegalArgumentException`.

PASO 4 Agent — un trabajador de la oficina (clase)

Atributos

Atributo	Tipo	Detalle
`id`	`String`	final. Identificador corto: `"dev"`
`name`	`String`	final. Nombre de persona: `"Ana"`
`role`	`Role`	final
`location`	`Location`	empieza en `DESK`
`memory`	`List<String>`	final. Empieza vacía (`new ArrayList<>()`)
`MAX_MEMORY`	`int`	`private static final int MAX_MEMORY = 5;`

Métodos

- Constructor `Agent(String id, String name, Role role)`.
- Getters: `getId()`, `getName()`, `getRole()`, `getLocation()`.
- `void moveTo(Location newLocation)` — cambia `location`.
- `void remember(String fact)` — añade `fact` al final de la memoria. Si después hay más de `MAX_MEMORY` elementos, **borra el más antiguo** (el primero). Es una memoria corta.
- `void receive(Message message)` — guarda en memoria el texto `<from> said: <content>`. Ejemplo: `ceo said: Your task: Design the logo`.
- `List<String> getMemory()` — devuelve `List.copyOf(memory)`.
- `String toString()` — devuelve `Ana (Developer)`. Usa `role.getDisplayName()`.

Pista
En un `ArrayList`, `memory.remove(0)` borra el primer elemento.

Hecho cuando…
después de llamar 6 veces a `remember(...)`, `getMemory()` tiene 5 elementos y el primero que guardaste ya no está.

PASO 5 Task — una tarea con estados (clase)

Una tarea pasa por tres estados y **solo puede avanzar en este orden**:

```
   assignTo(agent)        start()                complete(result)
PENDING ————————> PENDING ————————> IN_PROGRESS ————————————> DONE
(sin agente)        (con agente)
```

Atributos

Atributo	Tipo	Detalle
`id`	`String`	final. Ej. `"task-1"`
`title`	`String`	final. Ej. `"Design the logo"`
`role`	`Role`	final. El puesto que debe hacer la tarea
`status`	`TaskStatus`	empieza en `PENDING`
`assigneeId`	`String`	id del agente asignado. `null` hasta que se asigna
`result`	`String`	el resultado. `null` hasta que se completa

Métodos

- Constructor `Task(String id, String title, Role role)`. Si `title` es null o vacío → `IllegalArgumentException`.
- `void assignTo(Agent agent)`:
  - si la tarea no está en `PENDING` → `IllegalStateException`;
  - si el puesto del agente no coincide con el de la tarea → `IllegalArgumentException`;
  - si todo está bien, guarda `agent.getId()` en `assigneeId`.
- `void start()` — solo si está en `PENDING` **y** tiene agente asignado. Pasa a `IN_PROGRESS`. Si no se cumple → `IllegalStateException`.
- `void complete(String result)` — solo si está en `IN_PROGRESS`. Pasa a `DONE` y guarda el resultado. Si no → `IllegalStateException`.
- Getters de todos los atributos.

> **¿Qué excepción uso?**
> • `IllegalArgumentException`: me has pasado un **dato** incorrecto (un título vacío, un agente del puesto equivocado).
> • `IllegalStateException`: el **objeto** no está en el estado correcto para hacer eso (completar una tarea que no ha empezado).

> **Hecho cuando…**
> no hay forma de llamar a los métodos en un orden incorrecto sin que salte una excepción.

PASO 6 Office — la oficina (clase)

Atributos

Atributo	Tipo	Detalle
`agents`	`Map<String, Agent>`	final. Clave = id del agente. Usa `new LinkedHashMap<>()` (mantiene el orden en que contratas)
`tasks`	`List<Task>`	final
`messages`	`List<Message>`	final. Historial de todos los mensajes

Métodos

- `void hire(Agent agent)` — añade el agente. Si ya existe un agente con ese id → `IllegalArgumentException`.
- `Optional<Agent> findAgent(String id)` — busca por id.
- `Optional<Agent> findAgentByRole(Role role)` — devuelve el primer agente con ese puesto.
- `void addTask(Task task)`.
- `void send(Message message)` — si el destinatario (`to`) no existe → `IllegalArgumentException`. Si existe: guarda el mensaje en `messages` y llama a `receive(message)` del destinatario.
- `getAgents()`, `getTasks()`, `getMessages()` — devuelven copias inmutables (`List.copyOf(...)`; para el mapa, `List.copyOf(agents.values())`).

> **Pista**
> • `Optional` sirve para decir "puede que no haya resultado" sin devolver `null`. Para buscar por id: `Optional.ofNullable(agents.get(id))`.
> • Para buscar por puesto puedes recorrer `agents.values()` con un `for` y devolver `Optional.of(agent)` al encontrarlo, u `Optional.empty()` al final.

PASO 7 — LLMProvider y MockProvider — la IA falsa

Crea en el paquete `llm`:

- La interfaz `LLMProvider` con un único método: `String complete(String prompt);`
- La clase `MockProvider implements LLMProvider`:
  - constructor `MockProvider(List<String> responses)`;
  - cada llamada a `complete(...)` devuelve **la siguiente respuesta** de la lista, en orden. Ignora el prompt;
  - si ya no quedan respuestas → `IllegalStateException("No more mock responses")`.

> **¿Por qué una interfaz?**
> El resto del programa solo conocerá `LLMProvider`, nunca `MockProvider`. En la fase 2 crearás `GeminiProvider implements LLMProvider` y lo cambiarás en una sola línea del `main` sin tocar nada más. Esto se llama **programar contra una interfaz**.

> **Pista**
> Guarda la lista y un `int index` que empieza en 0. En cada llamada devuelves `responses.get(index)` y sumas 1.

PASO 8 — Simulation y ConsoleApp — juntarlo todo

Clase Simulation (paquete simulation)

- Dos atributos `final`, `Office office` y `LLMProvider llm`, que se reciben en el constructor.
- Método `void run(String order)`, que hace esto **en orden**:

1. Busca al agente con puesto `CEO` en la oficina. Imprime `[Client] -> Carla (CEO): <order>`. El CEO lo guarda en memoria con `remember("Client ordered: " + order)`.

2. Pide el plan a la IA: `llm.complete("Split this order into tasks: " + order)`.
3. Lee el plan: hay **una tarea por línea** con el formato `ROLE: title`. Ejemplo: `DESIGNER: Design the logo`. Por cada línea crea una `Task` con id `task-1`, `task-2`… y añádela a la oficina con `addTask`.
4. Si una línea no tiene `:` o el puesto no existe (ej. `JANITOR`), imprime un aviso y **sáltala**. El programa no debe romperse.
5. Para cada tarea: busca un agente de ese puesto → `assignTo` → el CEO le envía con `office.send(...)` el mensaje `Your task: <title>` → el agente va a `DESK` → `start()` → `result = llm.complete("Do this task: " + title)` → `complete(result)` → el agente envía al CEO `Done: <title>`.
6. Reunión final: todos los agentes van a `MEETING_ROOM`.
7. Imprime un resumen: cada tarea con su estado, quién la hizo y su resultado; y después la memoria de cada agente.

> **Pista para leer el plan**
> • `plan.split("\n")` separa por líneas.
> • `linea.split(":", 2)` separa en dos trozos como máximo: puesto y título. Usa `trim()` para quitar espacios.
> • `Role.valueOf("DESIGNER")` convierte el texto en el enum. Si el texto no es un puesto válido, lanza `IllegalArgumentException`: captúrala con `try/catch` para hacer el aviso del punto 4.

Clase ConsoleApp (paquete simulation)

Tiene un `public static void main(String[] args)` que:

1. Crea una `Office` y contrata 4 agentes, uno por puesto, con ids `ceo`, `dev`, `designer` y `tester`. Los nombres de persona los eliges tú.
2. Crea un `MockProvider` con estas 4 respuestas, en este orden (la línea `JANITOR` es incorrecta **a propósito**, para probar el punto 4):

```
List.of(
    "DEVELOPER: Build the home page\n"
        + "DESIGNER: Design the logo\n"
        + "TESTER: Test the home page\n"
        + "JANITOR: Clean the office",
    "index.html with a header and a menu",
    "logo.svg with a coffee cup",
    "Report: 0 bugs found"
)
```

3. Ejecuta `new Simulation(office, mock).run("Make a website for a coffee shop");`

Lo ejecutas con el **botón verde de Play** junto al `main` de `ConsoleApp`. No arranca Spring: es un programa Java normal.

Salida esperada (aproximada: no tiene que ser idéntica)

```
[Client] -> Carla (CEO): Make a website for a coffee shop
[Warning] Skipping invalid plan line: JANITOR: Clean the office
[Carla (CEO)] -> Ana (Developer): Your task: Build the home page
Ana (Developer) is working on: Build the home page
[Ana (Developer)] -> Carla (CEO): Done: Build the home page
...
Everyone is in the MEETING_ROOM

=== SUMMARY ===
task-1 [DONE] Build the home page | Ana (Developer) | index.html with a header...
task-2 [DONE] Design the logo | Leo (Designer) | logo.svg with a coffee cup
task-3 [DONE] Test the home page | Sara (Tester) | Report: 0 bugs found

=== MEMORIES ===
Carla (CEO): [Client ordered: Make a website..., dev said: Done: ..., ...]
Ana (Developer): [ceo said: Your task: Build the home page]
...
```

PASO 9 Tests con JUnit 5

Crea las clases de test en `src/test/java/com/marcosr_se/backend/` (paquetes `domain` y `llm`). Como mínimo, estos tests:

Clase	Tests (un método @Test cada uno)
`AgentTest`	`memoryKeepsOnlyLastFive` · `getMemoryCannotBeModified` · `receiveStoresMessageInMemory`
`TaskTest`	`newTaskIsPending` · `cannotStartUnassignedTask` · `cannotCompletePendingTask` · `assignToWrongRoleThrows` · `fullLifecycleEndsDone`
`OfficeTest`	`hireDuplicateIdThrows` · `sendDeliversToRecipientMemory` · `findAgentByRoleIsEmptyWhenMissing`
`MockProviderTest`	`returnsResponsesInOrder` · `throwsWhenNoMoreResponses`

Pista

• Cada test es un método con `@Test` encima. Comprueba resultados con `assertEquals(esperado, real)`, `assertTrue(...)` y `assertThrows(IllegalStateException.class, () -> task.start());`.
• `getMemoryCannotBeModified`: intenta hacer `agent.getMemory().add("x")` y comprueba que lanza `UnsupportedOperationException`.
• **No** pongas `@SpringBootTest` en estos tests: son Java puro y así se ejecutan en milisegundos.
• Para ejecutarlos: clic derecho en la carpeta `src/test/java` → **Run 'All Tests'**.

Hecho cuando…
todos los tests salen en verde.

Extras (opcionales, si te sobra tiempo)

- Añade a `Message` un cuarto componente `Instant sentAt` con la hora de envío.
- Haz que `Simulation` no use `System.out` directamente: crea una interfaz `SimulationListener` con `void onEvent(String event)` y que `ConsoleApp` le pase una que imprima. Esto prepara la fase 3, donde los eventos irán por WebSocket al navegador en vez de a la consola.
- Un test de `Simulation` con un `MockProvider` que compruebe que al final todas las tareas válidas están en `DONE`.

Checklist de entrega

- ☐ Los 9 pasos hechos, con un commit por paso (en inglés) y `git push` al final.
- ☐ `ConsoleApp` se ejecuta y la línea `JANITOR` muestra un aviso en vez de romper el programa.
- ☐ Todos los tests en verde.
- ☐ Ninguna clase de `domain` o `llm` importa nada de `org.springframework`.
- ☐ Ningún getter devuelve una lista interna (todas son `List.copyOf`).

> **Cuando te atasques**
> Pregúntame pegando **el error completo** y **el archivo** donde ocurre. No escribiré el código por ti, pero te explicaré qué pasa y por qué. Y cuando termines, pídeme una revisión antes del último push.